

二分查找算法完全指南

算法学习笔记

2026 年 5 月 12 日

目录

1	二分查找的核心概念	3
1.1	什么是二分查找？	3
1.2	为什么时间复杂度是 $O(\log n)$ ？	3
1.3	二分查找的关键——“二段性”	3
2	整数二分查找的两种经典模板	4
2.1	模板一：查找第一个满足条件的元素（左边界）	4
2.2	模板二：查找最后一个满足条件的元素（右边界）	5
2.3	为什么模板二需要 $mid = (l + r + 1) / 2$ ？	7
3	浮点数二分查找	7
4	STL 中的二分查找函数	9
5	深入理解：二段性与 check 函数	10
5.1	什么是好的 check 函数？	10
5.2	如何构造 check 函数？	10
6	经典例题详解	11
6.1	例题 1：在排序数组中查找元素的第一个和最后一个位置	11
6.2	例题 2：寻找峰值	12
6.3	例题 3：搜索旋转排序数组	14
6.4	例题 4：找第一个错误版本	15
6.5	例题 5：计算 x 的平方根	16
6.6	例题 6：在 D 天内完成所有任务	17

7 难题进阶——有深度的二分问题	19
7.1 进阶题 1: 寻找两个正序数组的中位数	19
7.2 进阶题 2: 分割数组的最大值	21
7.3 进阶题 3: 找出第 k 小的距离对	23
7.4 进阶题 4: 在排序矩阵中查找第 k 小的元素	25
8 二分答案——一种重要的解题思想	28
8.1 什么时候使用二分答案?	28
8.2 二分答案的通用框架	28
9 总结: 二分查找的核心要点	29
10 练习题推荐	29

1 二分查找的核心概念

1.1 什么是二分查找？

二分查找（Binary Search）是一种在有序序列中快速查找特定元素的算法。其核心思想是“减而治之”——每次都把搜索范围缩小一半。

形象理解

想象你在翻一本词典找某个单词：

1. 打开到中间位置
2. 如果目标单词应该在前面，就把后半部分丢掉
3. 如果目标单词应该在后面，就把前半部分丢掉
4. 重复这个过程，直到找到目标

1.2 为什么时间复杂度是 $O(\log n)$ ？

每次查找都能排除一半的数据：

- 第 1 次：剩 $n/2$ 个数据
- 第 2 次：剩 $n/4$ 个数据
- 第 3 次：剩 $n/8$ 个数据
- ...
- 第 k 次：剩 $n/2^k$ 个数据

当 $n/2^k = 1$ 时， $k = \log_2 n$ ，因此最多需要 $\log_2 n$ 次比较。

1.3 二分查找的关键——“二段性”

重要理解：二分查找的本质不是数据“有序”，而是数据具有“二段性”——我们可以用一个条件把数据分成“满足条件”和“不满足条件”的两段。

二段性示例

- 在有序数组中，以某个值为界，分为”小于目标值”和”大于等于目标值”
- 在旋转数组中，以旋转点为界，分为”较大的有序段”和”较小的有序段”
- 在峰值问题中，分为”在上升坡上”和”在下降坡上”

2 整数二分查找的两种经典模板

2.1 模板一：查找第一个满足条件的元素（左边界）

适用场景：找到第一个满足条件的元素位置

模板一核心记忆

”左边界四步法”：

1. 初始化： $l = 0, r = n - 1$
2. 循环条件： $while(l < r)$
3. 中点计算： $mid = (l + r) / 2$ （向下取整）
4. 判断更新：
 - 如果 mid 满足条件： $r = mid$ （不排除 mid ，向左收缩）
 - 如果 mid 不满足条件： $l = mid + 1$ （排除 mid ，向右走）

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      // 在有序数组arr中，查找第一个 >= x 的元素下标
6      // 如果不存在，返回 -1
7      int findFirstGreaterOrEqual(vector<int>& arr, int x) {
8          int l = 0, r = arr.size() - 1;
9
10         while (l < r) {
11             int mid = l + (r - l) / 2; // 等价于 (l + r) / 2，但
12             防止溢出
```

```

13         if (arr[mid] >= x) {
14             // mid满足条件 (>=x), 可能是答案, 往左找更小的
15             r = mid;
16         } else {
17             // mid不满足条件 (<x), mid肯定不是答案
18             l = mid + 1;
19         }
20     }
21
22     // while结束时 l == r, 这个位置就是答案
23     if (arr[l] >= x) return l;
24     return -1; // 所有元素都 < x
25 }
26
27 // 使用示例
28 int main() {
29     vector<int> arr = {1, 3, 3, 3, 5, 7, 9};
30     int x = 3;
31     int pos = findFirstGreaterOrEqual(arr, x);
32     cout << "第一个 >= " << x << " 的位置: " << pos << endl;
33     // 输出: 第一个 >= 3 的位置: 1
34     return 0;
35 }

```

Listing 1: 模板 1: 查找第一个 $\geq x$ 的元素

2.2 模板二: 查找最后一个满足条件的元素 (右边界)

适用场景: 找到最后一个满足条件的元素位置

模板二核心记忆

”右边界四步法”:

1. 初始化: $l = 0, r = n - 1$
2. 循环条件: $while(l < r)$
3. 中点计算: $mid = (l + r + 1) / 2$ (向上取整, 防止死循环)
4. 判断更新:
 - 如果 mid 满足条件: $l = mid$ (不排除 mid , 向右收缩)
 - 如果 mid 不满足条件: $r = mid - 1$ (排除 mid , 向左走)

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      // 在有序数组arr中, 查找最后一个 <= x 的元素下标
6      // 如果不存在, 返回 -1
7      int findLastLessOrEqual(vector<int>& arr, int x) {
8          int l = 0, r = arr.size() - 1;
9
10         while (l < r) {
11             int mid = l + (r - l + 1) / 2; // 注意: +1是关键, 防止死循环
12
13             if (arr[mid] <= x) {
14                 // mid满足条件 (<=x), 可能是答案, 往右找更大的
15                 l = mid;
16             } else {
17                 // mid不满足条件 (>x), mid肯定不是答案
18                 r = mid - 1;
19             }
20         }
21
22         if (arr[l] <= x) return l;
23         return -1; // 所有元素都 > x
24     }
25
26     // 使用示例
```

```

27     int main() {
28         vector<int> arr = {1, 3, 3, 3, 5, 7, 9};
29         int x = 3;
30         int pos = findLastLessOrEqual(arr, x);
31         cout << "最后一个 <= " << x << " 的位置: " << pos << endl
32             ;
33         // 输出: 最后一个 <= 3 的位置: 3
34         return 0;
35     }

```

Listing 2: 模板 2: 查找最后一个 $\leq x$ 的元素

2.3 为什么模板二需要 $mid = (l + r + 1)/2$?

这是为了防止死循环。考虑 $l = 0, r = 1$ 的情况:

死循环分析

- 如果使用 $mid = (l + r)/2$:
 - $mid = (0 + 1)/2 = 0$
 - 如果满足条件, 执行 $l = mid = 0$
 - 区间还是 $[0, 1]$, 没变化! 进入死循环
- 如果使用 $mid = (l + r + 1)/2$:
 - $mid = (0 + 1 + 1)/2 = 1$
 - 如果满足条件, 执行 $l = mid = 1$
 - 区间变成 $[1, 1]$, 退出循环

3 浮点数二分查找

浮点数二分不需要考虑整数边界问题, 只需要控制精度。

```

1     #include <iostream>
2     #include <cmath>
3     using namespace std;
4
5     // 计算x的平方根, 精确到1e-6
6     double sqrtBinary(double x) {

```

```

7      double l = 0, r = max(1.0, x); // 注意处理x<1的情况
8
9      // 方法1: 控制绝对误差
10     while (r - l > 1e-8) { // 精度要求
11         double mid = (l + r) / 2;
12         if (mid * mid >= x) {
13             r = mid;
14         } else {
15             l = mid;
16         }
17     }
18     return l;
19 }
20
21 // 方法2: 固定循环次数 (更常用)
22 double sqrtBinary2(double x) {
23     double l = 0, r = max(1.0, x);
24
25     for (int i = 0; i < 100; i++) { // 循环100次足够精确
26         double mid = (l + r) / 2;
27         if (mid * mid >= x) {
28             r = mid;
29         } else {
30             l = mid;
31         }
32     }
33     return l;
34 }
35
36 int main() {
37     double x = 2.0;
38     cout << "sqrt(2) = " << sqrtBinary(x) << endl;
39     cout << "实际值: " << sqrt(2) << endl;
40     // 输出: sqrt(2)    1.414214
41     return 0;
42 }

```

Listing 3: 浮点数二分: 求平方根

4 STL 中的二分查找函数

C++ STL 提供了现成的二分查找函数：

```
1      #include <iostream>
2      #include <algorithm>
3      #include <vector>
4      using namespace std;
5
6      int main() {
7          vector<int> arr = {1, 3, 3, 3, 5, 7, 9};
8
9          // 1. binary_search: 判断元素是否存在
10         bool exists = binary_search(arr.begin(), arr.end(), 3);
11         cout << "3是否存在: " << (exists ? "是" : "否") << endl;
12
13         // 2. lower_bound: 查找第一个 >= x 的位置 (左边界模板)
14         auto left = lower_bound(arr.begin(), arr.end(), 3);
15         cout << "第一个 >= 3 的位置: " << (left - arr.begin()) <<
16             endl;
17         cout << "对应的值: " << *left << endl;
18
19         // 3. upper_bound: 查找第一个 > x 的位置
20         auto right = upper_bound(arr.begin(), arr.end(), 3);
21         cout << "第一个 > 3 的位置: " << (right - arr.begin()) <<
22             endl;
23         cout << "对应的值: " << *right << endl;
24
25         // 4. equal_range: 同时获取左右边界
26         auto range = equal_range(arr.begin(), arr.end(), 3);
27         cout << "3的范围: [" << (range.first - arr.begin())
28             << ", " << (range.second - arr.begin()) << "]" << endl;
29
30         // 总结: 如果我们要找3出现的次数
31         int count = right - left;
32         cout << "3出现的次数: " << count << endl;
33
34         return 0;
35     }
```

Listing 4: STL 二分查找函数使用示例

5 深入理解：二段性与 check 函数

5.1 什么是好的 check 函数？

二分查找的核心是设计一个 **check** 函数，使得数据具有”二段性”：

check 函数设计原则

- 单调性：前面一段不满足，后面一段满足（或反之）
- 明确性：对于任意位置，能明确判断是否满足条件
- 目标性：check 函数的临界点就是我们想找的答案

5.2 如何构造 check 函数？

```
1 // 模式1：找第一个满足条件的（左边界）
2 // check函数：判断是否满足目标性质
3 bool check(vector<int>& arr, int mid, int target) {
4     return arr[mid] >= target; // 满足条件的性质
5 }
6
7 int binarySearchLeft(vector<int>& arr, int target) {
8     int l = 0, r = arr.size() - 1;
9     while (l < r) {
10         int mid = l + (r - l) / 2;
11         if (check(arr, mid, target)) {
12             r = mid; // 满足条件，往左找
13         } else {
14             l = mid + 1; // 不满足，往右找
15         }
16     }
17     return l;
18 }
19
20 // 模式2：找最后一个满足条件的（右边界）
21 int binarySearchRight(vector<int>& arr, int target) {
22     int l = 0, r = arr.size() - 1;
23     while (l < r) {
24         int mid = l + (r - l + 1) / 2;
25         if (check(arr, mid, target)) {
```

```

26         l = mid; // 满足条件，往右找
27     } else {
28         r = mid - 1; // 不满足，往左找
29     }
30 }
31 return l;
32 }

```

Listing 5: check 函数的设计模式

6 经典例题详解

6.1 例题 1：在排序数组中查找元素的第一个和最后一个位置

题目描述：给定一个升序排列的整数数组和一个目标值，找出目标值在数组中的开始位置和结束位置。

```

1     #include <iostream>
2     #include <vector>
3     using namespace std;
4
5     vector<int> searchRange(vector<int>& nums, int target) {
6         if (nums.empty()) return {-1, -1};
7
8         // 第一次二分：找第一个 >= target 的位置（左边界）
9         int l = 0, r = nums.size() - 1;
10        while (l < r) {
11            int mid = l + (r - l) / 2;
12            if (nums[mid] >= target) {
13                r = mid;
14            } else {
15                l = mid + 1;
16            }
17        }
18
19        // 检查是否找到了target
20        if (nums[l] != target) return {-1, -1};
21        int left_bound = l;
22
23        // 第二次二分：找最后一个 <= target 的位置（右边界）

```

```

24     r = nums.size() - 1; // l不变, 只重置r
25     while (l < r) {
26         int mid = l + (r - l + 1) / 2; // 注意+1
27         if (nums[mid] <= target) {
28             l = mid;
29         } else {
30             r = mid - 1;
31         }
32     }
33
34     return {left_bound, l};
35 }
36
37 int main() {
38     vector<int> nums = {5, 7, 7, 8, 8, 10};
39     int target = 8;
40     vector<int> result = searchRange(nums, target);
41
42     cout << "数组: ";
43     for (int num : nums) cout << num << " ";
44     cout << endl;
45     cout << "查找目标: " << target << endl;
46     cout << "范围: [" << result[0] << ", " << result[1] << "]"
47         << endl;
48     // 输出: [3, 4]
49
50     return 0;
51 }

```

Listing 6: 例题 1: 查找元素的起始和结束位置

6.2 例题 2: 寻找峰值

题目描述: 峰值元素是指其值大于左右相邻值的元素。给定一个数组，返回任意一个峰值元素的索引。

解题思路: 虽然数组无序，但具有二段性：

- 如果 $nums[mid] < nums[mid + 1]$ ，那么 mid 在上升坡上，峰值一定在右侧
- 否则， mid 在下降坡或峰顶，峰值在左侧（包括 mid ）

```

1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      int findPeakElement(vector<int>& nums) {
6          int l = 0, r = nums.size() - 1;
7
8          while (l < r) {
9              int mid = l + (r - l) / 2;
10
11              // 比较mid和mid+1, 判断在上升坡还是下降坡
12              if (nums[mid] > nums[mid + 1]) {
13                  // mid在下降坡, 峰值在左侧 (包括mid)
14                  r = mid;
15              } else {
16                  // mid在上升坡, 峰值在右侧 (不包括mid)
17                  l = mid + 1;
18              }
19          }
20
21          return l; // l就是峰值位置
22      }
23
24      int main() {
25          vector<int> nums = {1, 2, 1, 3, 5, 6, 4};
26          int peak = findPeakElement(nums);
27
28          cout << "数组: ";
29          for (int num : nums) cout << num << " ";
30          cout << endl;
31          cout << "峰值位置: " << peak << ", 峰值大小: " << nums[
                peak] << endl;
32          // 可能的输出: 峰值位置: 5, 峰值大小: 6
33
34          return 0;
35      }

```

Listing 7: 例题 2: 寻找峰值

6.3 例题 3：搜索旋转排序数组

题目描述：一个原本升序的数组在某个点进行了旋转，搜索目标值的位置。

例如：[4,5,6,7,0,1,2] 在原数组 [0,1,2,4,5,6,7] 的基础上旋转了 4 个位置。

解题思路：二段性体现在——总能找到一段是有序的，判断 target 是否在这段中。

```
1      #include <iostream>
2      #include <vector>
3      using namespace std;
4
5      int searchRotated(vector<int>& nums, int target) {
6          int l = 0, r = nums.size() - 1;
7
8          while (l <= r) { // 注意：这里是 <=
9              int mid = l + (r - l) / 2;
10
11             if (nums[mid] == target) {
12                 return mid; // 找到了
13             }
14
15             // 判断哪边是有序的
16             if (nums[l] <= nums[mid]) {
17                 // 左半部分有序
18                 if (nums[l] <= target && target < nums[mid]) {
19                     // target在左半部分
20                     r = mid - 1;
21                 } else {
22                     // target在右半部分
23                     l = mid + 1;
24                 }
25             } else {
26                 // 右半部分有序
27                 if (nums[mid] < target && target <= nums[r]) {
28                     // target在右半部分
29                     l = mid + 1;
30                 } else {
31                     // target在左半部分
32                     r = mid - 1;
33                 }
34             }
35         }
```

```

36
37     return -1; // 没找到
38 }
39
40 int main() {
41     vector<int> nums = {4, 5, 6, 7, 0, 1, 2};
42     int target = 0;
43
44     cout << "旋转数组: ";
45     for (int num : nums) cout << num << " ";
46     cout << endl;
47     cout << "查找目标: " << target << endl;
48     cout << "位置: " << searchRotated(nums, target) << endl;
49     // 输出: 位置: 4
50
51     return 0;
52 }

```

Listing 8: 例题 3: 搜索旋转排序数组

6.4 例题 4: 找第一个错误版本

题目描述: 有 n 个版本 $[1, n]$, 已知在某个版本之后都是错误的。用最少的调用次数找到第一个错误的版本。

```

1  #include <iostream>
2  using namespace std;
3
4  // 模拟的API: 判断版本是否正确
5  bool isBadVersion(int version) {
6      // 假设从版本4开始都是错误的
7      return version >= 4;
8  }
9
10 int firstBadVersion(int n) {
11     int l = 1, r = n;
12
13     while (l < r) {
14         int mid = l + (r - 1) / 2;
15
16         if (isBadVersion(mid)) {

```

```

17         // 当前版本是错误的，第一个错误版本可能是mid或更
           早
18         r = mid;
19     } else {
20         // 当前版本正确，第一个错误版本在后面
21         l = mid + 1;
22     }
23 }
24
25 return l; // l就是第一个错误版本
26 }
27
28 int main() {
29     int n = 10;
30     int bad = firstBadVersion(n);
31
32     cout << "总版本数: " << n << endl;
33     cout << "第一个错误版本: " << bad << endl;
34     // 输出: 第一个错误版本: 4
35
36     return 0;
37 }

```

Listing 9: 例题 4: 第一个错误版本

6.5 例题 5: 计算 x 的平方根

题目描述: 实现整数平方根函数, 计算并返回 x 的平方根, 只保留整数部分。

```

1     #include <iostream>
2     using namespace std;
3
4     int mySqrt(int x) {
5         if (x == 0) return 0;
6
7         int l = 1, r = x;
8
9         while (l < r) {
10             int mid = l + (r - l + 1) / 2; // 使用右边界模板
11
12             // 判断mid的平方是否 <= x

```



```

13         if (mid <= x / mid) { // 用除法避免溢出
14             // mid^2 <= x, 满足条件, 可能还有更大的答案
15             l = mid;
16         } else {
17             // mid^2 > x, 不满足, 答案在左边
18             r = mid - 1;
19         }
20     }
21
22     return l;
23 }
24
25 int main() {
26     int x1 = 8, x2 = 16, x3 = 2147395599;
27
28     cout << "sqrt(" << x1 << ") = " << mySqrt(x1) << endl;
29     // 2
30     cout << "sqrt(" << x2 << ") = " << mySqrt(x2) << endl;
31     // 4
32     cout << "sqrt(" << x3 << ") = " << mySqrt(x3) << endl;
33     // 46339
34
35     return 0;
36 }

```

Listing 10: 例题 5: x 的平方根（整数版本）

6.6 例题 6: 在 D 天内完成所有任务

题目描述: 传送带上有 n 个包裹, 第 i 个包裹重量为 $weights[i]$ 。要在 D 天内按顺序发送所有包裹, 求传送带的最小运载能力。

解题思路: 二分答案——能力值越大, 所需天数越少。找到第一个满足天数 $\leq D$ 的能力。这就是典型的”对答案进行二分”。

```

1     #include <iostream>
2     #include <vector>
3     #include <algorithm>
4     using namespace std;
5
6     // 计算以capacity的运载能力需要多少天
7     int daysNeeded(vector<int>& weights, int capacity) {

```

```

8      int days = 1; // 至少需要1天
9      int current_sum = 0;
10
11     for (int weight : weights) {
12         if (current_sum + weight > capacity) {
13             // 当前包裹放不下了，需要新的一天
14             days++;
15             current_sum = weight;
16         } else {
17             current_sum += weight;
18         }
19     }
20
21     return days;
22 }
23
24 int shipWithinDays(vector<int>& weights, int D) {
25     // 运载能力的范围：至少是最大包裹重量，最大是所有包裹总重量
26     int left = *max_element(weights.begin(), weights.end());
27     int right = 0;
28     for (int w : weights) right += w;
29
30     // 二分查找最小运载能力（左边界模板）
31     while (left < right) {
32         int mid = left + (right - left) / 2;
33
34         if (daysNeeded(weights, mid) <= D) {
35             // 以mid的运力能在D天内完成，说明mid可能太大了
36             // 尝试更小的运力
37             right = mid;
38         } else {
39             // 以mid的运力不能在D天内完成，必须增大运力
40             left = mid + 1;
41         }
42     }
43
44     return left;
45 }
46

```

```

47     int main() {
48         vector<int> weights = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
49         int D = 5;
50
51         int capacity = shipWithinDays(weights, D);
52         cout << "包裹重量: ";
53         for (int w : weights) cout << w << " ";
54         cout << endl;
55         cout << "需要在 " << D << " 天内完成" << endl;
56         cout << "最小运载能力: " << capacity << endl;
57         // 输出: 15
58
59         return 0;
60     }

```

Listing 11: 例题 6: 传送带运载能力

7 难题进阶——有深度的二分问题

7.1 进阶题 1: 寻找两个正序数组的中位数

题目描述: 给定两个大小分别为 m 和 n 的正序数组 $nums1$ 和 $nums2$ 。找出这两个正序数组的中位数，要求时间复杂度为 $O(\log(m+n))$ 。

解题思路: 这道题是二分的经典难题。核心思想是:

- 在较短的数组上二分查找“分割线”的位置
- 分割线左边元素个数 = 分割线右边元素个数（或左边多 1 个）
- 满足交叉大小关系: $nums1[i-1] \leq nums2[j]$ 且 $nums2[j-1] \leq nums1[i]$

```

1     #include <iostream>
2     #include <vector>
3     #include <algorithm>
4     #include <climits>
5     using namespace std;
6
7     double findMedianSortedArrays(vector<int>& nums1, vector<int>
8         & nums2) {
9         // 确保nums1是较短的数组，减少二分次数
10        if (nums1.size() > nums2.size()) {

```

```

10         return findMedianSortedArrays(nums2, nums1);
11     }
12
13     int m = nums1.size(), n = nums2.size();
14     int totalLeft = (m + n + 1) / 2; // 左半部分应该有多少个
        元素
15
16     // 在nums1上二分查找合适的分割线
17     int l = 0, r = m;
18
19     while (l < r) {
20         int i = l + (r - l + 1) / 2; // nums1的分割线位置
21         int j = totalLeft - i;       // nums2的分割线位置
22
23         if (nums1[i - 1] > nums2[j]) {
24             // nums1分割线左边太大了, 需要左移
25             r = i - 1;
26         } else {
27             // 满足条件, 尝试右移找更大的i
28             l = i;
29         }
30     }
31
32     int i = l; // nums1的分割位置
33     int j = totalLeft - i; // nums2的分割位置
34
35     // 处理边界情况
36     int nums1LeftMax = (i == 0) ? INT_MIN : nums1[i - 1];
37     int nums1RightMin = (i == m) ? INT_MAX : nums1[i];
38     int nums2LeftMax = (j == 0) ? INT_MIN : nums2[j - 1];
39     int nums2RightMin = (j == n) ? INT_MAX : nums2[j];
40
41     // 计算中位数
42     if ((m + n) % 2 == 1) {
43         // 奇数个元素, 中位数是左半部分最大值
44         return max(nums1LeftMax, nums2LeftMax);
45     } else {
46         // 偶数个元素, 中位数是左右部分最大最小值的平均
47         double left = max(nums1LeftMax, nums2LeftMax);
48         double right = min(nums1RightMin, nums2RightMin);

```

```

49         return (left + right) / 2.0;
50     }
51 }
52
53 int main() {
54     vector<int> nums1 = {1, 3};
55     vector<int> nums2 = {2};
56
57     cout << "数组 1: 1 3" << endl;
58     cout << "数组 2: 2" << endl;
59     cout << "中位数: " << findMedianSortedArrays(nums1, nums2
60         ) << endl;
61     // 输出: 2.0
62
63     vector<int> nums3 = {1, 2};
64     vector<int> nums4 = {3, 4};
65     cout << "\n数组 1: 1 2" << endl;
66     cout << "数组 2: 3 4" << endl;
67     cout << "中位数: " << findMedianSortedArrays(nums3, nums4
68         ) << endl;
69     // 输出: 2.5
70
71     return 0;
72 }

```

Listing 12: 进阶题 1: 两个有序数组的中位数

7.2 进阶题 2: 分割数组的最大值

题目描述: 给定一个非负整数数组和一个整数 m ，将数组分成 m 个非空的连续子数组。设计一个算法使得这 m 个子数组各自和的最大值最小。

解题思路:

- 二分答案: 子数组和的最大值在 $[\max(nums), \sum(nums)]$ 之间
- check 函数: 判断能否在不超过该最大和的情况下分成 $\leq m$ 个子数组
- 找左边界: 第一个能分成 $\leq m$ 组的最大和

```

1     #include <iostream>
2     #include <vector>

```

```

3      #include <algorithm>
4      #include <numeric>
5      using namespace std;
6
7      // 判断在最大和为maxSum的情况下，能否分成 <= m 个子数组
8      bool canSplit(vector<int>& nums, int m, int maxSum) {
9          int count = 1; // 当前分成的子数组个数
10         int currentSum = 0;
11
12         for (int num : nums) {
13             if (currentSum + num > maxSum) {
14                 // 当前子数组加不下这个数了，需要新的一组
15                 count++;
16                 currentSum = num;
17                 if (count > m) return false;
18             } else {
19                 currentSum += num;
20             }
21         }
22
23         return true;
24     }
25
26     int splitArray(vector<int>& nums, int m) {
27         // 二分范围：至少是最大元素，至多是所有元素之和
28         long long left = *max_element(nums.begin(), nums.end());
29         long long right = accumulate(nums.begin(), nums.end(), 0
30             LL);
31
32         // 找左边界：第一个能分成 <= m 组的最大和
33         while (left < right) {
34             long long mid = left + (right - left) / 2;
35
36             if (canSplit(nums, m, mid)) {
37                 // mid可行，尝试更小的最大值
38                 right = mid;
39             } else {
40                 // mid太小了，分成的组数 > m，需要增大
41                 left = mid + 1;
42             }
43         }
44     }
45 }

```

```

42     }
43
44     return left;
45 }
46
47 int main() {
48     vector<int> nums = {7, 2, 5, 10, 8};
49     int m = 2;
50
51     cout << "数组: 7 2 5 10 8" << endl;
52     cout << "分割成 " << m << " 份" << endl;
53     cout << "最小的最大子数组和: " << splitArray(nums, m) <<
54         endl;
55     // 输出: 18 (分为 [7,2,5] 和 [10,8])
56
57     return 0;
58 }

```

Listing 13: 进阶题 2: 分割数组的最大值最小化

7.3 进阶题 3: 找出第 k 小的距离对

题目描述: 给定一个整数数组, 返回所有数对之间的绝对差值中第 k 小的值。

解题思路:

- 首先排序数组
- 二分答案: 差值在 $[0, \max - \min]$ 之间
- check 函数: 计算差值 $\leq mid$ 的数对个数 (用双指针或二分)
- 找第一个满足“差值 $\leq mid$ 的数对个数 $\geq k$ ”的值

```

1     #include <iostream>
2     #include <vector>
3     #include <algorithm>
4     using namespace std;
5
6     // 计算差值 <= mid 的数对个数
7     int countPairs(vector<int>& nums, int mid) {
8         int count = 0;
9         int n = nums.size();

```

```

10
11 // 双指针法，对于每个左指针i，找到最远的右指针j
12 for (int i = 0, j = 0; i < n; i++) {
13     while (j < n && nums[j] - nums[i] <= mid) {
14         j++;
15     }
16     // 对于固定的i，有j-i-1个数满足条件
17     count += j - i - 1;
18 }
19
20 return count;
21 }
22
23 int smallestDistancePair(vector<int>& nums, int k) {
24     // 先排序
25     sort(nums.begin(), nums.end());
26
27     int n = nums.size();
28     int left = 0, right = nums[n - 1] - nums[0];
29
30     // 二分查找第一个满足 countPairs(mid) >= k 的值
31     while (left < right) {
32         int mid = left + (right - left) / 2;
33
34         if (countPairs(nums, mid) >= k) {
35             // 差值 <= mid 的数对已经 >= k，说明第k小的差值
36             // <= mid
37             right = mid;
38         } else {
39             // 差值 <= mid 的数对不够k个，第k小的差值 > mid
40             left = mid + 1;
41         }
42     }
43
44     return left;
45 }
46
47 int main() {
48     vector<int> nums = {1, 3, 1};
49     int k = 1;

```



```

49
50     cout << "数组: 1 3 1" << endl;
51     cout << "第 " << k << " 小的距离对: "
52     << smallestDistancePair(nums, k) << endl;
53     // 所有距离对: (1,3)=2, (1,1)=0, (3,1)=2
54     // 排序后: 0, 2, 2, 第1小是0
55
56     nums = {1, 1, 1};
57     k = 2;
58     cout << "\n数组: 1 1 1" << endl;
59     cout << "第 " << k << " 小的距离对: "
60     << smallestDistancePair(nums, k) << endl;
61     // 所有距离对都是0, 第2小是0
62
63     nums = {1, 6, 1};
64     k = 3;
65     cout << "\n数组: 1 6 1" << endl;
66     cout << "第 " << k << " 小的距离对: "
67     << smallestDistancePair(nums, k) << endl;
68     // 距离对: (1,6)=5, (1,1)=0, (6,1)=5
69     // 排序后: 0, 5, 5, 第3小是5
70
71     return 0;
72 }

```

Listing 14: 进阶题 3: 第 k 小的距离对

7.4 进阶题 4: 在排序矩阵中查找第 k 小的元素

题目描述: 给定一个 $n \times n$ 矩阵, 每行和每列元素均按升序排列。找到矩阵中第 k 小的元素。

解题思路:

- 二分答案: 第 k 小元素在 $[matrix[0][0], matrix[n-1][n-1]]$ 之间
- check 函数: 计算矩阵中 $\leq mid$ 的元素个数
- 利用矩阵的有序性, 在 $O(n)$ 时间内完成计数

```

1     #include <iostream>
2     #include <vector>

```

```

3      using namespace std;
4
5      // 计算矩阵中 <= mid 的元素个数
6      int countLessEqual(vector<vector<int>>& matrix, int mid) {
7          int n = matrix.size();
8          int count = 0;
9
10         // 从左下角开始统计
11         int row = n - 1, col = 0;
12
13         while (row >= 0 && col < n) {
14             if (matrix[row][col] <= mid) {
15                 // 当前列的所有上方元素都 <= mid
16                 count += row + 1;
17                 col++; // 向右移动
18             } else {
19                 row--; // 向上移动
20             }
21         }
22
23         return count;
24     }
25
26     int kthSmallest(vector<vector<int>>& matrix, int k) {
27         int n = matrix.size();
28         int left = matrix[0][0];
29         int right = matrix[n - 1][n - 1];
30
31         // 找左边界：第一个满足 countLessEqual(mid) >= k 的值
32         while (left < right) {
33             int mid = left + (right - left) / 2;
34
35             if (countLessEqual(matrix, mid) >= k) {
36                 right = mid;
37             } else {
38                 left = mid + 1;
39             }
40         }
41
42         return left;

```

```

43     }
44
45     int main() {
46         vector<vector<int>> matrix = {
47             {1, 5, 9},
48             {10, 11, 13},
49             {12, 13, 15}
50         };
51         int k = 8;
52
53         cout << "矩阵:" << endl;
54         for (auto& row : matrix) {
55             for (int num : row) cout << num << " ";
56             cout << endl;
57         }
58         cout << "第 " << k << " 小的元素: "
59         << kthSmallest(matrix, k) << endl;
60         // 输出: 13
61
62         matrix = {{1, 2}, {1, 3}};
63         k = 2;
64         cout << "\n矩阵:" << endl;
65         for (auto& row : matrix) {
66             for (int num : row) cout << num << " ";
67             cout << endl;
68         }
69         cout << "第 " << k << " 小的元素: "
70         << kthSmallest(matrix, k) << endl;
71         // 输出: 1
72
73         return 0;
74     }

```

Listing 15: 进阶题 4: 排序矩阵中的第 k 小元素

8 二分答案——一种重要的解题思想

8.1 什么时候使用二分答案？

二分答案的典型场景

当一个问题满足以下条件时，可以尝试二分答案：

1. 答案具有单调性：答案越大（或越小），越容易满足条件
2. 可以快速验证：对于给定的答案，能在较短时间内判断是否可行
3. 求最值：通常求“最大值的最小值”或“最小值的最大值”

8.2 二分答案的通用框架

```
1 // 假设我们要找"满足条件的最小值"
2 int binaryAnswer(long long left, long long right) {
3     while (left < right) {
4         long long mid = left + (right - left) / 2; // 左边界
           模板
5
6         if (check(mid)) {
7             // mid满足条件，尝试更小的值
8             right = mid;
9         } else {
10            // mid不满足条件，需要更大的值
11            left = mid + 1;
12        }
13    }
14    return left; // 最小的满足条件的答案
15 }
16
17 // check函数：判断某个答案是否可行
18 bool check(long long answer) {
19     // 根据问题要求，判断以answer作为答案是否满足条件
20     // 返回true表示满足，false表示不满足
21 }
```

Listing 16: 二分答案的通用模板

9 总结：二分查找的核心要点

必须掌握的关键点

1. 理解二段性：二分查找的核心不是“有序”，而是能找到一种属性将数据分成两段
2. 模板选择：
 - 找左边界（第一个满足条件的）： $mid = (l + r) / 2$, $r = mid$, $l = mid + 1$
 - 找右边界（最后一个满足条件的）： $mid = (l + r + 1) / 2$, $l = mid$, $r = mid - 1$
3. 防止死循环：使用右边界模板时， mid 计算必须 $+1$
4. 防止溢出：使用 $mid = l + (r - l) / 2$ 而不是 $(l + r) / 2$
5. 二分答案：很多最优解问题可以通过二分答案转化为判定问题
6. 灵活运用：二分不限于数组，可以二分下标、二分值域、二分时间等

10 练习题推荐

1. 基础训练：

- 查找插入位置（LeetCode 35）
- 找出有序数组中的单一元素（LeetCode 540）
- 寻找比目标字母大的最小字母（LeetCode 744）

2. 变形训练：

- 搜索旋转排序数组 II（LeetCode 81）
- 寻找旋转排序数组中的最小值（LeetCode 153）
- 找出峰值（LeetCode 162）

3. 二分答案：

- 吃香蕉的速度（LeetCode 875）
- 分割数组的最大值（LeetCode 410）
- 制作 m 天所需的花束（LeetCode 1482）

4. 困难挑战:

- 寻找两个正序数组的中位数 (LeetCode 4)
- 找出第 k 小的距离对 (LeetCode 719)
- 有序矩阵中第 K 小的元素 (LeetCode 378)
- 最小化最大值 (LeetCode 410)
- 青蛙过河 (LeetCode 403)